3 庞加莱截面

3.1 绘制思路

对于一个混沌系统，其轨线往往是混乱无序的，很难去简洁的描述。那么其中一个思路就是，简化信息，降低描述的维度，把空间的轨线化简为一系列离散点。

对于一个周期或拟周期运动的系统，在相空间的运动表现为一圈又一圈的转动。我们定义一个截面（一般是平面），当轨线穿过这个面时，把交点记录下来。当记录足够多的交点后，这些交点形成的图像就是庞加莱截面的图像。我们把这个和轨线相交的面称作**庞加莱截面**（Poincaré section）。

接下来就以杜芬（Duffing）方程分岔图的绘制大概演示一下上面所说的大概思路。

Duffing方程是以 Georg Duffing命名的一个非线性方程。它是基于强迫振动的单摆所提出的方程，它提出的时间非常早，但是被拿来做混沌研究还是比较晚的。由于它背后有着非常明显与简单的物理模型，所以甚至可以做实验去观察这个方程的非线性[6]。方程的形式为：

$$\ddot{x}=-x^3+x-\delta\dot{x}+\gamma\cos\left(\omega t\right)$$

把这个方程整理成一个三阶的方程组，得到：

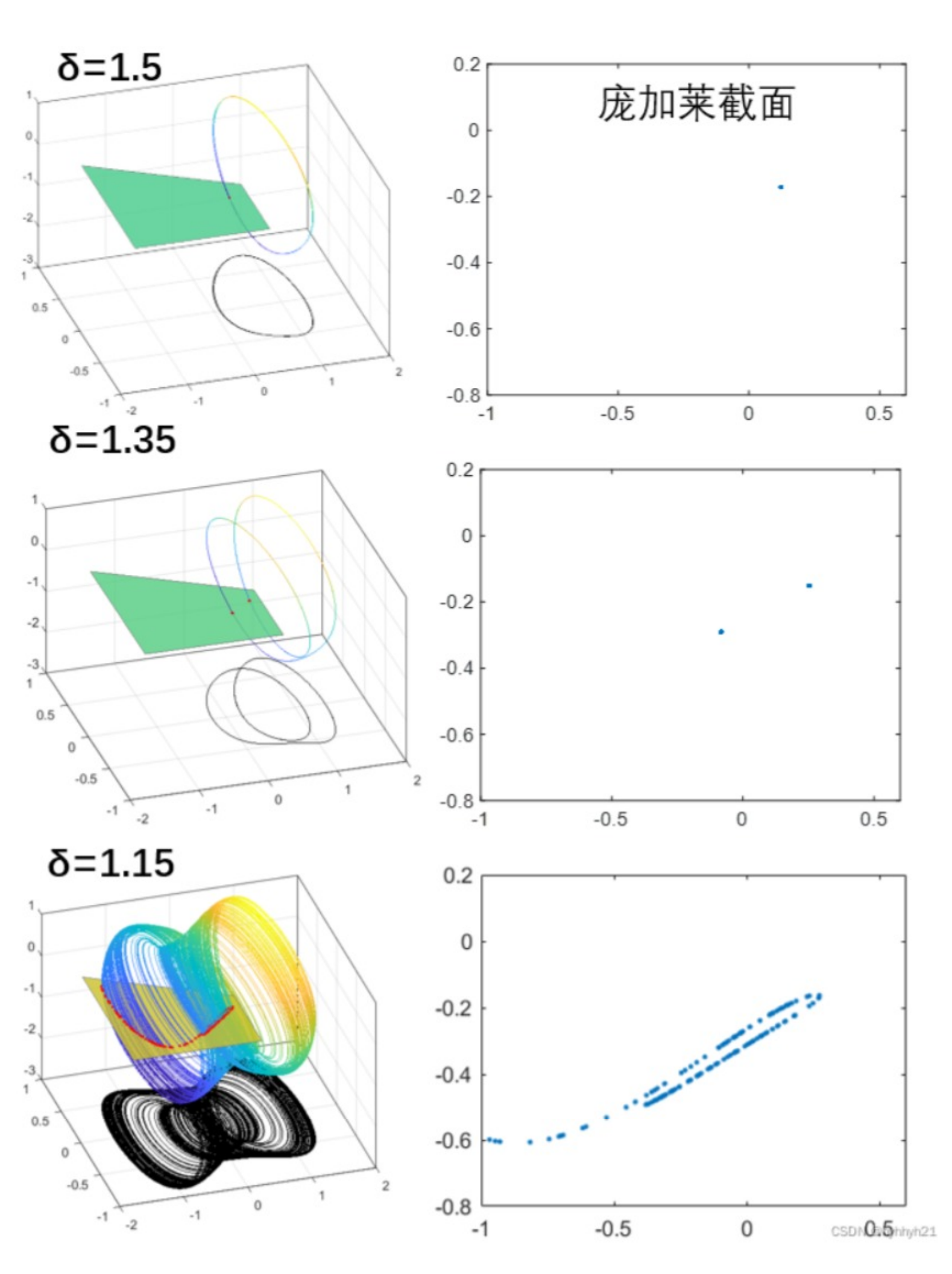
$$\begin{bmatrix}x\\ \dot{x}\\ z\end{bmatrix}'=\begin{bmatrix}\dot{x}\\ -x^3+x-\delta\dot{x}+\gamma z\\ -\omega\sin\left(\omega t\right)\end{bmatrix}$$

这里有个随时间变化的变量cos(wt)，为了方便分析，把它单独定义为z=cos(wt)。

现在，我就得到了一个三阶的非线性方程组。默认ω=1，γ=1。

之后，我们定义空间中的z=0作为其要切割轨线的平面。而且当轨线正向穿过平面时才被记录，反向穿过平面则不被记录。这是为了让一个周期只取记录1个点。

下图是当δ=1.5、δ=1.35、δ=1.15时，其三维轨线和庞加莱截面：



当然，实际上定义的z=0截面应该是空间上没有边界限制无穷大的，这里为了方便作图，所以用了一个四边形来代替，希望不要造成误解。

通过庞加莱截面，就可以知道系统的大概特性。比如当δ=1.5时，庞加莱截面是只有1个点，证明系统是周期1的系统。当δ=1.35时，庞加莱截面是变为2个点，说明系统是周期2的系统，每转两次回到起点。当δ=1.5时，庞加莱截面是变为接近一条曲线，说明系统很可能是一个准周期运动。

所以庞加莱截面的绘制方法可以分为3步：

```
1 计算出轨线
2 计算轨线与某一平面的交点
3 将交点绘制出来，就得到了庞加莱截面
```

其中第二步计算交点，通常采用插值的方法得到。

这里给出绘图代码：

```matlab
%庞佳莱截面
%截面采用公式Ax+By+Cz+D=0;的形式
%采用杜芬方程
clear
clc
close all
%% 第一步，计算出轨迹
h=5e-3;
x0=0:h:1600;
y0=[0.1;0.1;1];%最后一项是cos(w*t)，当t=0时必须为1.
[y1,Output]=ODE_RK4_hyh(x0,h,y0,[1.5,1,1]);
%[1.5,1,1],[1.35,1,1],[1.15,1,1]
Lx=y1(1,2000:end);
Ly=y1(2,2000:end);
Lz=y1(3,2000:end);
%% 第二步，计算与平面的交点
Plane=[0;0;1;0];%一般情况下是个垂直某个轴的平面。这里是Ax+By+Cz+D=0的形式给出的平面。
[tP_List,yP_List]=Solve_Poincare(x0,y1,Plane);%计算Poincare平面

%% 第三步，绘制庞加莱截面
%1庞加莱截面
%最开始几个点还没有稳定，没有体现出系统特点，所以放弃
figure()
plot(yP_List(1,10:end),yP_List(2,10:end),'.')%从第10个点之后开始绘制
xlim([-1,0.6])
ylim([-0.8,0.2])

%2展示用的示意图，只作为演示
figure()
hold on
patch([Lx,nan],[Ly,nan],[Lz,nan],[Lx+Ly,nan],...
    'EdgeColor','interp','Marker','none','MarkerFaceColor','flat','LineWidth',0.8,'FaceAlpha',1);
plot3(yP_List(1,10:end),yP_List(2,10:end),zeros(size(yP_List(2,10:end))),...
    '.','MarkerSize',8,'color','r')
patch([-1.6,0.4,0.4,-1.6],[-0.7,-0.7,0.0,0.6],[0,0,0,0],[1,1,1,1],...
    'FaceAlpha',0.8,'EdgeColor',[0.5,0.5,0.5])
view([-17,39])
box on
grid on
%绘制投影的相图
set(gcf,'position',[300 200 560 500])
xlim([-2,2])
zlim([-3,1])
plot3( Lx,Ly,zeros(size(Ly))-3 ,'color','k')
hold off

%% 后置函数
function [tP_List,yP_List]=Solve_Poincare(t,y,Plane)
%截面方程z=0
% Plane=[0;0;1;0];%一般情况下是个垂直某个轴的平面
%一般只记录从负到正穿越。如果想反向也记录，可以设置Plane=-Plane。

%第二步，插值得到线与面的交点
yP_List=[];
tP_List=[];
Dis=DistancePlane(y,Plane);
N=size(y,2);
for k=1:N-1
    if Dis(k)<=0 && Dis(k+1)>0
        t0=t(k);t1=t(k+1);
        yP0=y(:,k);yP1=y(:,k+1);
        Dis0=Dis(k);Dis1=Dis(k+1);
        %一维线性插值，求Dis=0时的t和y
        %（相比较前面积分的4阶RK，这里用线性插值精度有点低，先这么凑合着吧）
        yP=yP0+(yP1-yP0)/(Dis1-Dis0)*(0-Dis0);
        tP=t0+(t1-t0)/(Dis1-Dis0)*(0-Dis0);
        %储存
        yP_List=[yP_List,yP];
        tP_List=[tP_List,tP];
    end
end
end


%点到平面的距离
function Dis=DistancePlane(xk,Plane)
% xk，坐标点，如果是3维坐标，大小就是3*N的矩阵。
% Plane，平面，形如Ax+By+Cz+D=0形式的平面。

N=size(xk,2);%计算总共多少个点
xk2=[xk;ones(1,N)];
Dis=dot(xk2,Plane*ones(1,N),1)./norm(Plane(1:end-1));
end

%两点线性插值
function y=interp2point_linear(x0,x1,y0,y1,x)
y=y0+(y1-y0)/(x1-x0)*(x-x0);
end

%两点3次插值
function y=interp2point_spline(x0,x1,y0,y1,x)
%y0包含y0的值和y0的导数,yy=y0(1),dy=y0(2)
xx0=x0;
xx1=x1;
yy0=y0(1);dy0=y0(2);
yy1=y1(1);dy1=y1(2);
cs = csape([xx0,xx1],[dy0,yy0,yy1,dy1],[1,1]);
y=ppval(cs,x);
end

function [F,Output]=Fdydx(x,y,Input)
%形式为Y'=F(x,Y)的方程，参见数值分析求解常系数微分方程相关知识
%高次用列向量表示，F=[dy(1);dy(2)];y(1)为函数，y(2)为函数导数
%杜芬方程duffing，参见中国大学MOOC，北京师范大学-计算物理基础-77例摆与杜芬方程
d=Input(1);
r=Input(2);
w=Input(3);

dy(1)=y(2);
dy(2)=-y(1)^3+y(1)-d*y(2)+r*y(3);
dy(3)=-w*sin(w*x);

F=[dy(1);dy(2);dy(3)];
Output=[];
end

function [y,Output]=ODE_RK4_hyh(x,h,y0,Input)
%4阶RK方法
%h间隔为常数的算法
y=zeros(size(y0,1),size(x,2));
y(:,1)=y0;
for ii=1:length(x)-1
    yn=y(:,ii);
    xn=x(ii);
    [K1,~]=Fdydx(xn    ,yn       ,Input);
    [K2,~]=Fdydx(xn+h/2,yn+h/2*K1,Input);
    [K3,~]=Fdydx(xn+h/2,yn+h/2*K2,Input);
    [K4,~]=Fdydx(xn+h  ,yn+h*K3  ,Input);
    y(:,ii+1)=yn+h/6*(K1+2*K2+2*K3+K4);
end
Output=[];
end
```

3.2 利用频闪法快速绘制

这种计算方法，具体思路还是那三步。

```
1 计算出轨线
2 计算轨线与某一平面的交点
3 将交点绘制出来，就得到了庞加莱截面
```

但是第二步不再是直接通过轨线上两点进行插值得到，而是直接通过时间得到。

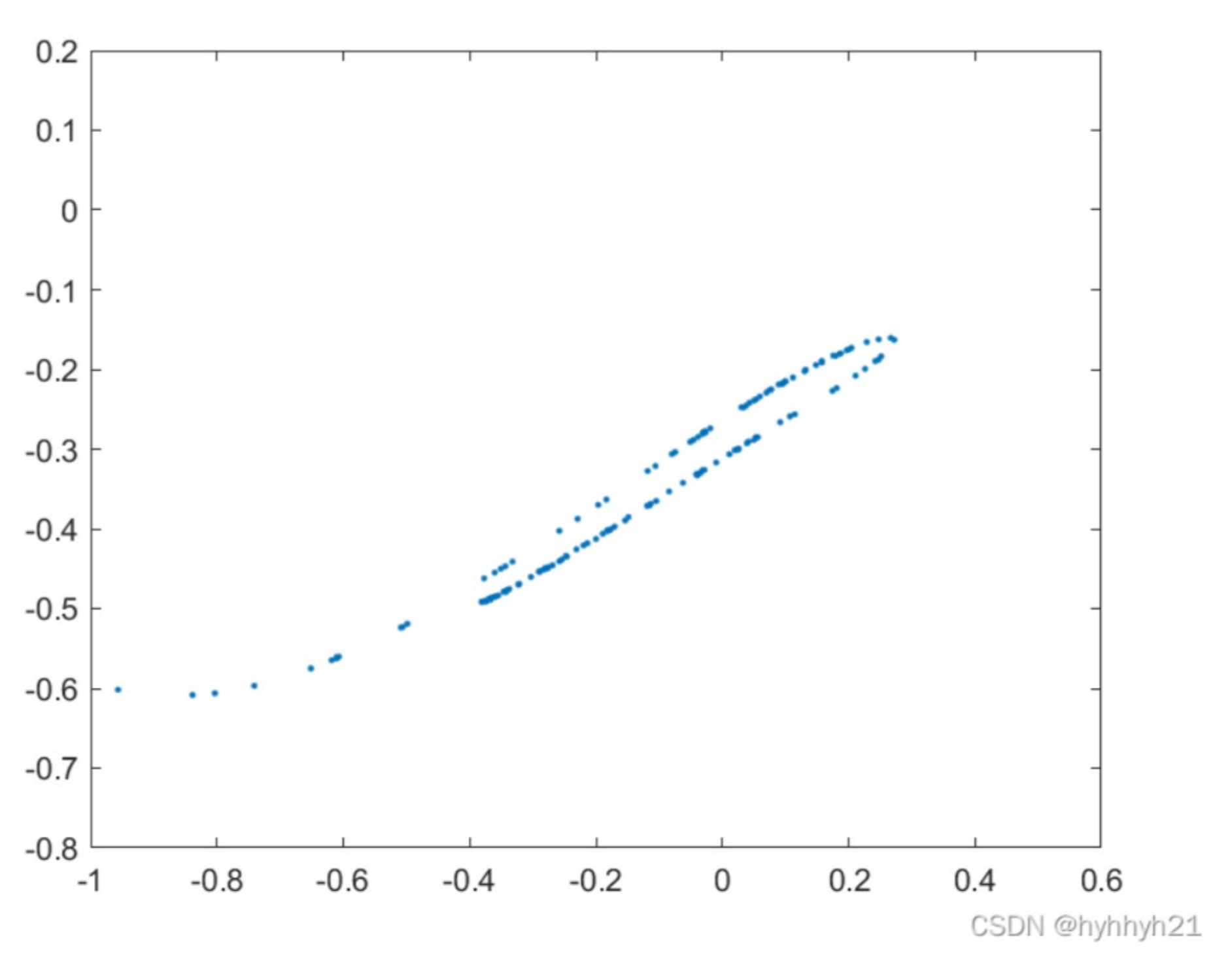
比如下面是我们3.1节的杜芬方程，我们取的是z=0平面作为庞加莱截面。

$$\begin{bmatrix}x\\ \dot{x}\\ z\end{bmatrix}'=\begin{bmatrix}\dot{x}\\ -x^3+x-\delta\dot{x}+\gamma z\\ -\omega\sin\left(\omega t\right)\end{bmatrix}$$

但是，z=0明明是可以计算出来的。z=cos(t)，当t=0.5π+2kπ和t=1.5π+2kπ的时候，z无条件的等于0。
对于庞加莱截面，一个周期我们只取一个点，所以所有位于庞加莱截面上的点，其时间t都应该满足t=1.5π+2kπ。

因为，对于频闪采样法，意思就是隔一段时间取一次点，这些点天然的在某个庞加莱截面上。这种方法采用预先计算得到的时刻间隔，大大减少了程序内的计算量。

同样取δ=1.15，其庞加莱截面的结果如下图。可以看到这种偷懒方法和上一章节老老实实计算结果相同。



事实上，由于不需要再计算z具体的值，方程中甚至可以省略掉计算z的步骤。这样就变成了虽然没有z这变量，但是却依然计算了系统在z=0的庞加莱截面。

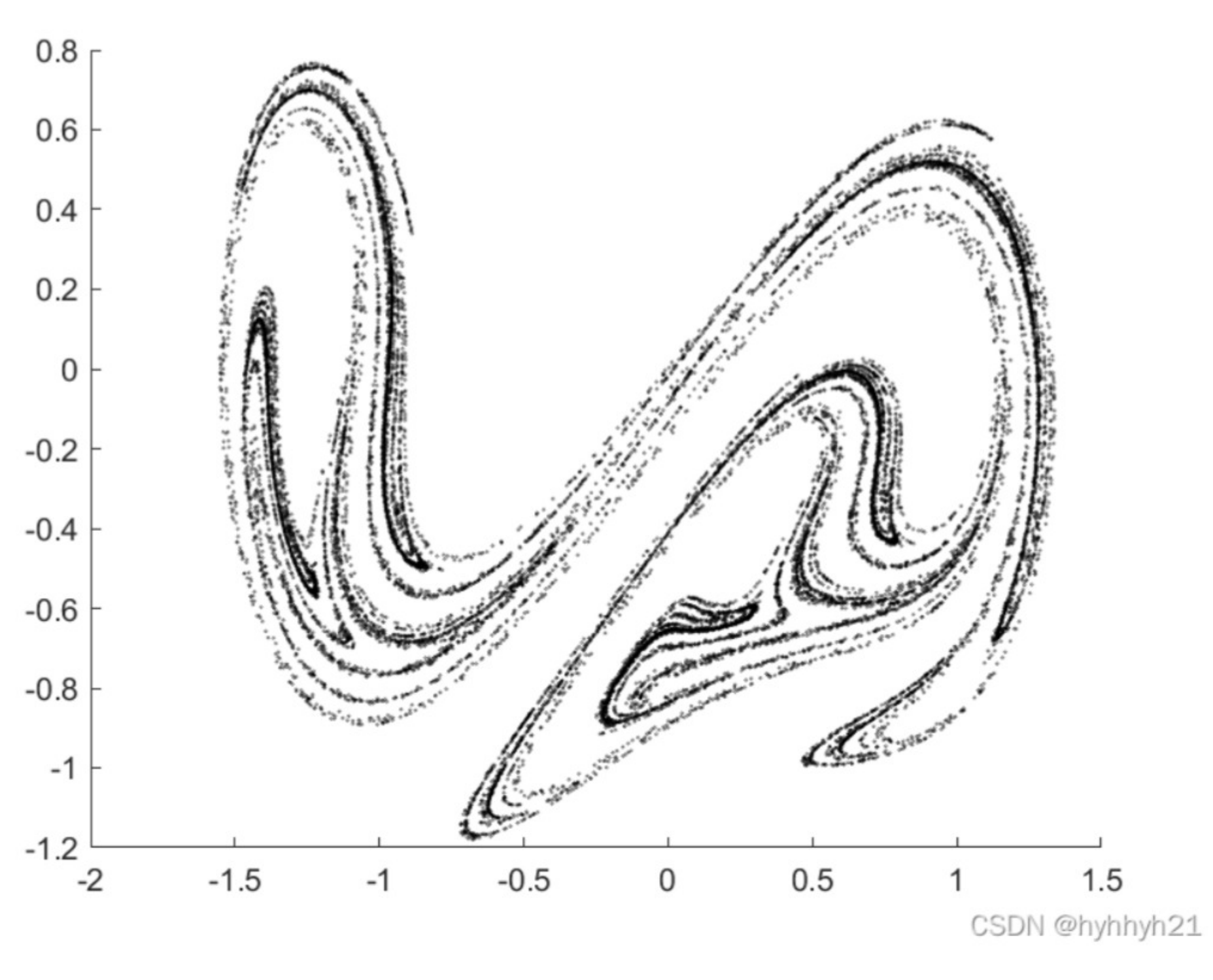
$$\begin{bmatrix}x\\ \dot{x}\end{bmatrix}'=\begin{bmatrix}\dot{x}\\ -x^3+x-\delta\dot{x}+\gamma\cos\left(\omega t\right)\end{bmatrix}$$

代码如下：

```matlab
%庞佳莱截面（频闪法）
%采用杜芬方程
clear
clc
close all
%第一步，计算出轨迹
h=5e-3;
x0=0:h:1600;
y0=[0.1;0.1];%这里简化了方程，所以只有两个输入
[y1,Output]=ODE_RK4_hyh(x0,h,y0,[1.15,1,1]);
%[1.5,1,1],[1.35,1,1],[1.15,1,1],[0.1,0.35,1.4]
Lx=y1(1,:);
Ly=y1(2,:);
%Lz=cos(1*x0);

%采用频闪采样法计算
tP_Ideal=3*pi/2:(2*pi/1):x0(end);%这里考虑z=0平面，时间间隔dt=2*pi。
tP_List=zeros(1,length(tP_Ideal));
Ind_List=zeros(1,length(tP_Ideal));
for k=1:length(tP_Ideal)
    [~,Ind]=min(abs( tP_Ideal(k)-x0 ));%直接根据索引来找到对应的点
    Ind_List(k)=Ind;
    tP_List(k)=x0(Ind);
end
yP_List=y1(:,Ind_List);
%注，如果上面时间间隔h为pi/600之类的形式，这里连min函数都可以取消，直接按照索引去寻找就行

%绘图
%1庞加莱截面
%最开始几个点还没有稳定，没有体现出系统特点，所以放弃
figure()
plot(yP_List(1,10:end),yP_List(2,10:end),'.')
xlim([-1,0.6])
ylim([-0.8,0.2])


function [F,Output]=Fdydx(x,y,Input)
%形式为Y'=F(x,Y)的方程，参见数值分析求解常系数微分方程相关知识
%高次用列向量表示，F=[dy(1);dy(2)];y(1)为函数，y(2)为函数导数
d=Input(1);
r=Input(2);
w=Input(3);

dy(1)=y(2);
dy(2)=-y(1)^3+y(1)-d*y(2)+r*cos(w*x);
% dy(3)=-w*sin(w*x);%由于无需计算具体的z值，所以这里把z合并到了dy(2)项里，减少计算量

F=[dy(1);dy(2)];
Output=[];
end

function [y,Output]=ODE_RK4_hyh(x,h,y0,Input)
%4阶RK方法
%h间隔为常数的算法
y=zeros(size(y0,1),size(x,2));
y(:,1)=y0;
for ii=1:length(x)-1
    yn=y(:,ii);
    xn=x(ii);
    [K1,~]=Fdydx(xn    ,yn       ,Input);
    [K2,~]=Fdydx(xn+h/2,yn+h/2*K1,Input);
    [K3,~]=Fdydx(xn+h/2,yn+h/2*K2,Input);
    [K4,~]=Fdydx(xn+h  ,yn+h*K3  ,Input);
    y(:,ii+1)=yn+h/6*(K1+2*K2+2*K3+K4);
end
Output=[];
end
```

这种方法只在某些特殊的存在明确周期性的系统中才可以用的到。

当[δ，γ，ω] = [0.1 ,0.35 ,1.4]时，系统会由于混沌，其庞加莱截面还会出现混沌形式：



上面的图涉及到的点非常多，所以无法一口气全部得到，只能采取一段一段计算的方式得到。绘图代码如下，仅供参考：

```matlab
clear
clc
close all
%初始化
h=2*pi/700;
x0=0:h:2e5*pi/700;
y0=[0.1;0.1];%最后一项是cos(w*t)，当t=0时必须为1
%这里一口气求出一大堆点不太现实，内存有限，所以采取分批计算，逐步绘图来实现
figure(1)
hold on
for k=1:100
    [y1,Output]=ODE_RK4_hyh(x0,h,y0,[0.1,0.35,1.4]);
    Lx=y1(1,:);
    Ly=y1(2,:);
    %采用频闪采样法计算
    %由于w变为了1.4，这里还是z=0平面，采样间隔改为 (3*pi/2:(2*pi/1):2000)/1.4
    yP_List=y1(:,376:500:end);%注，这里376=3/2/1.4*350+1

    %内存有点不够了，分段进行下面的计算
    x0=x0+2e5*pi/700;%时间整体向后移动
    y0=y1(:,end);%把最后一刻的状态当做下一个时间段初始状态
    %绘图
    %1庞加莱截面
    scatter(yP_List(1,1:end),yP_List(2,1:end),0.8,'k','MarkerEdgeAlpha',0.6)
    %disp(k)
end

function [F,Output]=Fdydx(x,y,Input)
%形式为Y'=F(x,Y)的方程，参见数值分析求解常系数微分方程相关知识
%高次用列向量表示，F=[dy(1);dy(2)];y(1)为函数，y(2)为函数导数
d=Input(1);
r=Input(2);
w=Input(3);
dy(1)=y(2);
dy(2)=-y(1)^3+y(1)-d*y(2)+r*cos(w*x);
F=[dy(1);dy(2)];
Output=[];
end

function [y,Output]=ODE_RK4_hyh(x,h,y0,Input)
%4阶RK方法
%h间隔为常数的算法
y=zeros(size(y0,1),size(x,2));
y(:,1)=y0;
for ii=1:length(x)-1
    yn=y(:,ii);
    xn=x(ii);
    [K1,~]=Fdydx(xn    ,yn       ,Input);
    [K2,~]=Fdydx(xn+h/2,yn+h/2*K1,Input);
    [K3,~]=Fdydx(xn+h/2,yn+h/2*K2,Input);
    [K4,~]=Fdydx(xn+h  ,yn+h*K3  ,Input);
    y(:,ii+1)=yn+h/6*(K1+2*K2+2*K3+K4);
end
Output=[];
end
```

4 非线性系统的分岔图

前面介绍了离散系统绘制分岔图。对于连续系统，如何绘制分岔图呢？

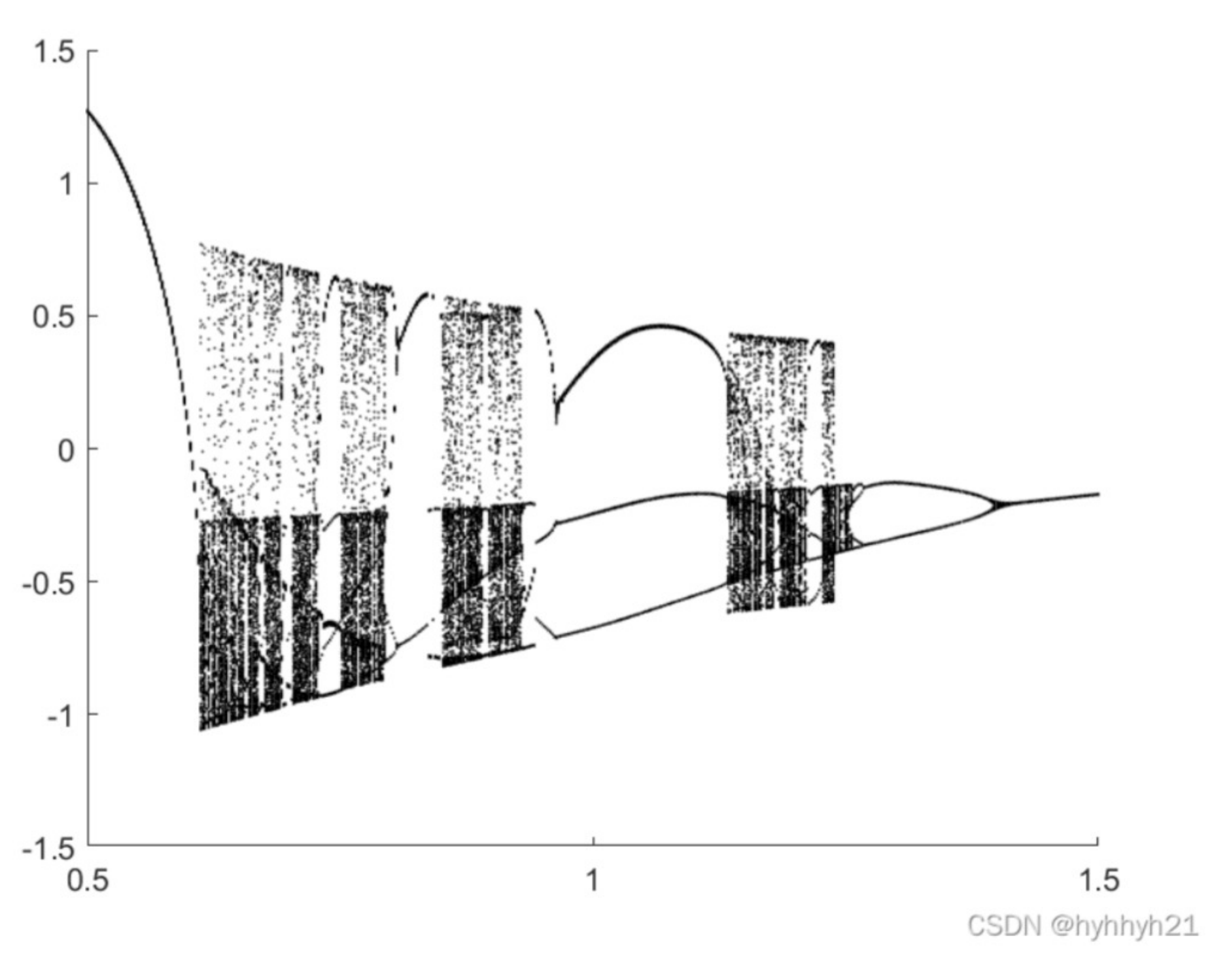
大概思路为：

```
 1先绘制出系统的轨线 （3维系统转3维曲线，用轨线）
 2绘制轨线和某个面的交点（3维曲线转2维点，用庞加莱截面）
得到某个平面的上的若干点后，问题就转化为前面二维离散系统的分岔图了：
 4将这些点投影到某个线上（2维的点转1维的值）
 5绘制分岔图
```

目的就是把连续系统转化为可以绘制分岔图的散点形式。

4.1 Duffing系统分岔图

以前面的介绍的杜芬方程为例，固定ω=1，γ=1，改变δ的分岔图如下。绘制方法就是计算出每一个δ下的庞加莱截面，然后仿照前面离散点分岔的，取坐标y作为分岔图。



绘图代码如下，计算时间比较长，需要耐心等待一段时间：

```matlab
%利用庞佳莱截面绘制分岔图
clear
clc
close all
%第一步，计算出轨迹
h=8e-3;
x0=0:h:1600;
y0=[0.1;0.1];%最后一项是cos(w*t)，当t=0时必须为1.

%不同的d
d=0.5:0.002:1.5;%0.02
N_c=length(d);
N_P=250;%假设穿过截面的共有250个点
BF=nan(N_c,N_P);
%第二步，这里直接用频闪法求出截面所在点的索引
%z=0平面
tP_Ideal=3*pi/2:(2*pi/1):x0(end);%这里考虑z=0平面，时间间隔dt=2*pi。
tP_List=zeros(1,length(tP_Ideal));
Ind_List=zeros(1,length(tP_Ideal));
for m=1:length(tP_Ideal)
    [~,Ind]=min(abs( tP_Ideal(m)-x0 ));
    Ind_List(m)=Ind;
    tP_List(m)=x0(Ind);
end
%第三步，开始对每一个d进行循环，计算其庞加莱截面
for k=1:N_c
    d_k=d(k);
    [y1,~]=ODE_RK4_hyh(x0,h,y0,[d_k,1,1]);
    yP_List=y1(:,Ind_List);
    %储存
    N_P_temp=size(tP_List,2);
    if N_P_temp>N_P
        BF(k,1:N_P)=yP_List(2,1:N_P);%取坐标y作为分岔图
    else
        BF(k,1:N_P_temp)=yP_List(2,1:N_P_temp);%取坐标y作为分岔图
    end
    disp(d_k)
end
%最后一步，绘图
figure()
hold on
for k=1:N_c
    d_k=d(k);
    plot(d_k*ones(1,N_P-30+1),BF(k,30:N_P),...
        'LineStyle','none','Marker','.','MarkerFaceColor','k','MarkerEdgeColor','k',...
        'MarkerSize',1)
end
hold off

function [F,Output]=Fdydx(x,y,Input)
%形式为Y'=F(x,Y)的方程，参见数值分析求解常系数微分方程相关知识
%高次用列向量表示，F=[dy(1);dy(2)];y(1)为函数，y(2)为函数导数
d=Input(1);
r=Input(2);
w=Input(3);
dy(1)=y(2);
dy(2)=-y(1)^3+y(1)-d*y(2)+r*cos(w*x);
F=[dy(1);dy(2)];
Output=[];
end

function [y,Output]=ODE_RK4_hyh(x,h,y0,Input)
%4阶RK方法
%h间隔为常数的算法
y=zeros(size(y0,1),size(x,2));
y(:,1)=y0;
for ii=1:length(x)-1
    yn=y(:,ii);
    xn=x(ii);
    [K1,~]=Fdydx(xn    ,yn       ,Input);
    [K2,~]=Fdydx(xn+h/2,yn+h/2*K1,Input);
    [K3,~]=Fdydx(xn+h/2,yn+h/2*K2,Input);
    [K4,~]=Fdydx(xn+h  ,yn+h*K3  ,Input);
    y(:,ii+1)=yn+h/6*(K1+2*K2+2*K3+K4);
end
Output=[];
end
```